

Module 15) Advance Python Programming

5. Exception Handling

- **Introduction to exceptions and how to handle them using try, except, and finally**

What are Exceptions?

- Exceptions are **errors that occur during program execution**.
- They disrupt the normal flow of a program.
- Examples include division by zero, accessing an undefined variable, or trying to open a non-existent file

Why Handle Exceptions?

- To **prevent the program from crashing** unexpectedly.
- To provide **meaningful error messages**.
- To allow the program to **recover or fail gracefully**.

How to Handle Exceptions?

Python uses **try, except, and finally** blocks for exception handling:

1. try block

- You put the code that **might raise an exception** inside a try block.
- Python attempts to execute this code.

2. except block

- If an exception occurs in the try block, execution jumps to the corresponding except block.
- You can specify the type of exception to catch or catch all exceptions.
- It handles the error so the program continues running.

3. finally block

- The code inside finally **always runs**, regardless of whether an exception occurred or not.
- It is typically used for cleanup actions like closing files or releasing resources.

- **Understanding multiple exceptions and custom exceptions.**

1. Handling Multiple Exceptions

Sometimes, a block of code can raise different types of exceptions. Python allows you to handle multiple exceptions in a few ways:

2. Custom Exceptions

Sometimes built-in exceptions are not enough to describe errors specific to your application. You can **create your own exception classes** by inheriting from Python's built-in Exception class (or its subclasses).

Handling multiple exceptions with one except block:

```
try:
```

```
except (IOError, ValueError) as e:
```

```
    print(f"An error occurred: {e}")
```

Creating a custom exception:

```
class MyCustomError(Exception):
```

```
    pass
```